



S.G. Ganesh

Understanding 'Side-effects' in C/C++ Programs

Even experienced C/C++ developers don't entirely understand the side effects of operators. We cover this important topic this month.

I get mails on technical topics from readers with various backgrounds. A simple question from M Rajasekar prompted me to write about 'side-effects' in this column. He asked: "What is the output of the following program?"

```
int a=5;
printf("%d %d %d",a++a--,++a);
```

As a student, I remember seeing such questions while writing written tests in campus placements (sometimes they are even asked in interviews). It just indicates how even experienced engineers who prepare the question paper, do not understand the question (nor do they know the answer!).

Here is a short answer to the question: the program has 'implementation defined behaviour'. The longer answer is: the expressions depend on 'side effects', and the resulting value of evaluating such expressions cannot be reliably predicted since the result can change with implementations.

So what are 'side-effects'? As the name indicates, they are other changes caused because of the evaluation of a main computation. In the statement "b = a++", the main computation is the assignment of the value of *a* to *b*; the incremented value of *a* (because of *a++*) will be available only in the next statement and this is a 'side-effect'. If the statement is "a = a++" it is a problem. Should the incremented value of *a* be (re)assigned to *a*, or should that result be lost? The C/C++ standards leave it to the implementation. Why?

To understand that, we need to know how an expression such as *a++* is implemented. While generating code, compilers put the value of *a* in a register and the temporary value of *a++* in another register. Once the current statement ends, the temporary value is assigned back to *a*. Now, for an expression such as 'a = a++', compilers can use different approaches. One compiler might use the register where the original value of *a* was stored; another compiler might use the register where the temporary value was stored. So, depending on the 'implementation' of the compiler, the result differs. C language gives this freedom to the compiler because the compiler can make use of this implementation choice to create the most efficient code for the given platform.

Let us look at an example to understand the unintuitive nature of programs that depend on side effects.

```
signed char ch = 5;
while(ch = ch--)
    printf("%d",ch);
```

You can get the output as 43210, 4321 or even an infinite loop, printing 555555... depending on your 'implementation'!

To determine the impact of 'side effects' in an expression, C/C++ standards define 'sequence points'. Even experts find it difficult to understand the intricacies of sequence points, so we won't dare attempt to cover that topic here.

How about languages like Java and C#? As you might know, Java and C# do not have any 'implementation defined behaviour', which is good. For example, if the initial value of *a* is 2, after executing the statement "a = a++", the value of *a* is 2; the program has a 'well-defined behaviour'.

Let us not debate whether the C/C++ approach or the Java/C# approach is better. It does not matter if the behaviour is implementation defined or well-defined—what is clear is that the behaviour is difficult to understand. If the initial value of *a* is 2, I initially expected that the value of *a* will be 3 after executing "a = a++" in Java/C#. So, programs that depend on side effects are unintuitive and hard to understand. Don't ever write such code.

So what do you do if you're asked such a question in a written test or interview? Well, I don't know! Maybe send a mail to their HR team informing them that the question is 'silly, stupid and rubbish', though I guarantee that you won't get any response (and certainly not a job from them)! Better still, don't use their software—it's going to be as bad as their written test or interview questions! 

About the author:

S G Ganesh is a research engineer in Siemens (Corporate Technology). His latest book is "60 Tips on Object Oriented Programming", published by Tata McGraw-Hill. You can reach him at sgganesh@gmail.com.